

Musterlösung — Übungsklausur 4

Deep Learning 1 · TU Berlin

1 Multiple Choice

1. Receptive Field?

✓ (b) Eingabegebiet, auf das ein Neuron in der Feature Map reagiert

- (a) Kanalanzahl
- (c) Kernelgröße
- (d) Pooling-Schichten

2. Max Pooling?

✓ (a) Translationsinvariant innerhalb des Pooling-Fensters

- (b) Erhöht Auflösung
- (c) Lernbare Parameter
- (d) Equivariance

3. XAI Ziel?

✓ (b) Vorhersagen interpretierbar und erklärbar machen

- (a) Hohe Testgenauigkeit
- (c) Trainingszeit
- (d) Adversarielle Robustheit

4. Shapley Values?

✓ (b) Beitrag jedes Input-Features zur Vorhersage

- (a) Gewichte
- (c) Aktivierungswerte
- (d) Gradienten

5. Forget Gate LSTM?

✓ (b) Steuert, welche Information aus dem Cell State gelöscht wird

- (a) Exploding Gradients
- (c) Output-Mapping
- (d) Gewichtsinitialisierung

6. Distribution Shift?

✓ (b) Die wahre Verteilung kann sich über die Zeit verändern

- (a) Batch-Verschiebung
- (c) Augmentierung
- (d) Normalisierung

7. Bidirektionale RNNs?

✓ (a) Verarbeiten Sequenz zweimal: vorwärts + rückwärts

- (b) Keine Zukunftsinformation
- (c) Weniger Parameter
- (d) Nur Klassifikation

8. 0/1-Loss Problem?

✓ (b) Gradient fast überall null → nicht via GD optimierbar

- (a) Differenzierbar aber sensitiv
- (c) Penalisiert korrekte Vorhersagen
- (d) Nur Multiclass

2 Theorie — LSTM Gates

(a) Gate-Funktion

$$\text{gate} = \sigma(W_g \cdot [x_t, h_{t-1}] + b_g) \in (0,1)^n$$

Eine Gate-Funktion ist mathematisch eine elementweise Multiplikation mit einer Sigmoid-Aktivierung, die Werte in (0,1) produziert: 0 = geschlossen, 1 = offen.

(b) Drei Gates

Forget Gate f_g : 'Erase'-Operation — $c_t = f_g \blacksquare c_{t-1}$. Entscheidet, welche Information aus dem Cell State gelöscht wird.

Input Gate i_g : 'Write'-Operation — $c_t += i_g \blacksquare \tanh(W \cdot [x, h])$. Entscheidet, welche neue Information in den Cell State geschrieben wird.

Output Gate o_g : 'Read'-Operation — $h_t = o_g \blacksquare \tanh(c_t)$. Entscheidet, welche Information aus dem Cell State an den Hidden State weitergegeben wird.

(c) Gradientenfluss durch Cell State

Der Cell State c wird standardmäßig nicht durch nichtlineare Funktionen oder Gewichtsmatrizen transformiert — er wird nur durch elementweise Multiplikation (Forget Gate) oder Addition (Input Gate) modifiziert.

Gradienten fließen additiv durch die Zeitschritte: $\partial c_t / \partial c_{t-1} \approx f_g$ (elementweise, nicht durch W multipliziert). Das vermeidet das Vanishing-Gradient-Problem des einfachen RNN, wo W^T wiederholt in den Gradienten eingeht.

3 Theorie — Gewichtsinitialisierung

(a) Herleitung Xavier-Initialisierung

Annahmen: $E[W^{(l)}_{ij}] = 0$, Inputs $z^{(l)}_j$ unkorreliert, gleichmäßige Varianz.

$$\begin{aligned} \text{Var}(z^{(l+1)}_i) &= \text{Var}(\sum_j W_{ij} \cdot z^{(l)}_j) = \sum_j \text{Var}(W_{ij}) \cdot \text{Var}(z^{(l)}_j) \\ &= n \cdot \eta \cdot \text{Var}(z^{(l)}_j) \end{aligned}$$

Setze $\eta = 1/n$, dann: $\text{Var}(z^{(l+1)}_i) = \text{Var}(z^{(l)}_j)$ — Varianz bleibt erhalten über alle Schichten.

(b) Xavier-Initialisierung

$$W^{(l)}_{ij} \sim N(0, 1/n) \text{ mit } n = \text{Anzahl Inputs der Schicht}$$

(c) Biases initial null

Biases auf 0 setzen stellt sicher, dass im ersten Forward Pass alle Neuronen symmetrisch aktiviert werden. Die Symmetriebrechung erfolgt durch zufällige Gewichte, nicht durch Biases. Nonzero-Biases würden zudem die Herleitung der optimalen Gewichtsvarianz verkomplizieren.

4 Theorie — Energy-Based Learning

(a) Push-Pull Training

Ziehe (Pull) die Energie für korrekte Ziel-Output-Paare (x, y_i) nach unten.

Schiebe (Push) die Energie für 'falsche' Antworten (kontrastive Beispiele) nach oben.

Resultat: korrekte Outputs haben minimale Energie, falsche höhere Energie $\rightarrow$ Modell wählt $\arg \min_y E(x, y)$.

(b) Generalized Perceptron Loss

$$L_{\text{perc}}(Y^i, E(W, X^i)) = E(W, Y^i, X^i) - \min_{\{Y \in \mathcal{Y}\}} E(W, Y, X^i)$$

Ungeeignet für Generalisierung: Margin = 0. Der Loss ist 0 sobald die korrekte Antwort die geringste Energie hat, egal wie knapp. Kein 'Safety Margin' → das Modell lernt keine robuste Trennung.

(c) Hinge Loss vs. Perceptron Loss

Hinge Loss: $L_{\text{hinge}} = \max(0, \Delta + E(W, Y^i, X^i) - E(W, \hat{Y}, X^i))$ mit Hyperparameter $\Delta > 0$.

Vorteil: Erzwingt einen Margin Δ zwischen korrekter und bester falscher Antwort. Das Modell muss korrekte Antworten nicht nur minimal besser, sondern um mindestens Δ besser einordnen → bessere Generalisierung.

5 Programmierung — LSTM Sequenzklassifikation

(a) LSTM-Klasse

```
class LSTMClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=10, hidden_size=64, batch_first=True)
        self.fc = nn.Linear(64, 5)
    def forward(self, x): # x: (batch, seq_len, 10)
        out, (h_n, c_n) = self.lstm(x)
        return self.fc(h_n[-1]) # letzter Hidden State
```

(b) Initialisierung h und c

nn.LSTM initialisiert h und c standardmäßig mit Nullen (wenn nicht übergeben).

Strategie 1: Nullinitialisierung (Standard, praktisch). Strategie 2: Lernbare Initialisierung (h , c als nn.Parameter). Strategie 3: Zufällige Initialisierung (Netz muss sich desensibilisieren).